

Digital Legal Assistant Chatbot

This project is our graduation project for the Digital Pioneers of Egypt Initiative (DEPI). It is a specialized chatbot designed to support law students and legal professionals in studying and researching the Egyptian Constitution.

Team Members

- Mostafa Ahmed Elgendy
- Mahmoud Ahmed Salah El-Din Abbas
- Ahmed Ebrahim Elmesery
- Abdullah El-Sayed Saad Hafz
- Hager Ayman Abdullah

Project Overview

This project is an AI-powered chatbot designed to help law students, lawyers, and legal researchers interact with the Egyptian Constitution in a modern and efficient way. The system allows users to upload a PDF version of the Constitution, automatically process its content, and then ask questions related to any article or topic within the document. Using advanced embedding models and a vector database, the chatbot retrieves the most relevant information and provides accurate, context-aware answers in Arabic.

Stakeholder Analysis

The key stakeholders of this project include law students, lawyers, the project development team, academic supervisors, and the Digital Pioneers of Egypt Initiative (DEPI). Law students and legal professionals represent the primary users of the chatbot, as they rely on it to study and search within the Egyptian Constitution. The project team and supervisors play a critical role in developing, guiding, and evaluating the system. DEPI acts as the main sponsor and has high influence over the project's direction and standards.

Problem Statement

This project addresses several key challenges faced by law students and legal professionals when studying the Egyptian Constitution. Traditional methods of searching through the Constitution are time-consuming, especially when dealing with long documents or looking for specific articles. Many users struggle to quickly locate relevant information or understand how different articles relate to one another. Additionally, there is a lack of modern, accessible digital tools that provide fast, accurate, and interactive access to constitutional content. The chatbot solves these problems by enabling users

to upload the Constitution, search instantly, and receive clear, AI-powered responses that make studying and legal research more efficient and accessible.

Milestones

Week 1: Requirements validation, PDF ingestion prototype, environment setup.

Week 2: Vector database integration, retrieval QA tests, error handling.

Week 3: Frontend chat experience, history sidebar, upload workflow polish.

Week 4: Integration tests, deployment scripts, documentation pass.

Tools

- We use the “paraphrase-multilingual-MiniLM-L12-v2” model to convert text into 384-dimensional semantic embeddings, enabling accurate similarity search
- The embedding vectors are stored in ChromaDB, which supports fast approximate nearest neighbor
- For retrieval, we compute the query embedding, search ChromaDB for the most similar text chunks, and use those as context for answer generation.
- We apply a Retrieval-Augmented Generation (RAG) pipeline: the retrieved document chunks are combined with the user query and sent to an LLM (Google Gemini) to generate context-aware responses.
- To process the Constitution PDF, we use a library (PyMuPDF) to extract the raw text, then split it into smaller “chunks” before embedding.
- The backend is built using a web framework (FastAPI) to handle file uploads, embedding generation, vector search, LLM calls, and chat history.
- The frontend is built using HTML, CSS, JS and bootstrap

UI/UX Design

- Arabic-first layout with mirrored alignment (right-to-left)
- Minimal cognitive load: clear upload area, focused chat pane, contrasting colors for user vs bot messages.
- Accessibility: readable font sizes, keyboard navigation, status indicators for processing states.

User Flows

1. First-time setup: user opens page, uploads constitution PDF, waits for confirmation, sees ready state message.
2. Ask question: users send queries, submit, observe streaming response with cited articles.

3. History recall: click past question to re-run or view cached answer; highlight selected entry.